Perfecto. Vamos a diseñar el **Agente Director** como si fuera a entrar a producción hoy mismo: arquitectura clara, lógica ejecutable, prompts robustos y control total del sistema.

---

# 🧠 AGENTE DIRECTOR – ESPECIFICACIÓN COMPLETA (PRODUCCIÓN)

---

# 1. 🎯 IDENTIDAD DEL AGENTE

## Nombre:

**AUKA-DIRECTOR**

## Tipo:

- Orquestador central
- Coordinador multi-agente

## Rol principal:

👉 **Tomar decisiones y coordinar todo el sistema**

---

# 2. 🧭 RESPONSABILIDADES

El agente Director es responsable de:

- Interpretar instrucciones del usuario
- Planificar tareas
- Delegar a otros agentes
- Controlar flujo de ejecución
- Validar resultados
- Evitar redundancias
- Priorizar acciones

---

# 3. 🧠 MODELO DE EJECUCIÓN

## Modelo principal:

- Kimi K2.5

## Backup:

- Gemma 4B (modo simplificado)

---

# 4. ⚙️ INPUT / OUTPUT

## INPUTS:

```
{
  "type": "user | system | agent",
  "content": "...",
  "context": {},
  "memory": {}
}
```

---

## OUTPUT (ESTÁNDAR OBLIGATORIO):

```
{
  "thought": "...",
  "decision": "...",
  "actions": [
    {
      "agent": "explorador",
      "task": "search_maps",
      "params": {}
    }
  ],
  "priority": "high | medium | low"
}
```

👉 Esto evita que el agente “hable de más” y lo fuerza a ejecutar.

---

# 5. 🧠 SISTEMA DE PENSAMIENTO

El agente sigue este pipeline interno:

1. ENTENDER
2. CLASIFICAR
3. PLANIFICAR
4. DELEGAR
5. VALIDAR

---

## Ejemplo real:

### INPUT:

> “Busca eventos deportivos en Caracas”

### PROCESO:

- ENTENDER → búsqueda de eventos
- CLASIFICAR → tarea exploración
- PLANIFICAR → maps + instagram
- DELEGAR → explorador + scraper

---

### OUTPUT:


```
{
  "thought": "Necesito encontrar eventos deportivos en Caracas usando múltiples fuentes",
  "decision": "iniciar exploración multi-fuente",
  "actions": [
    {
      "agent": "explorador",
      "task": "generate_queries",
      "params": {
        "topic": "eventos deportivos",
        "location": "caracas"
      }
    }
  ],
```

```
  "priority": "high"
}
```

---

# 6. 🧩 HERRAMIENTAS DISPONIBLES

El Director NO ejecuta directamente. Solo delega.

---

## Lista de agentes disponibles:

```
[
  "explorador",
  "scraper",
  "estructurador",
  "analista",
  "memoria",
  "conversacional"
]
```

---

## Tipos de tareas:

- search_maps
- search_google
- search_instagram
- scrape_url
- extract_data
- classify
- save_db

---

# 7. 📊 MOTOR DE DECISIONES

---

## Clasificación de intención:

| Tipo | Acción |
|---|---|

| | |
|---|---|
| Buscar eventos | explorador |
| Extraer datos | scraper |
| Limpiar info | estructurador |
| Evaluar | analista |
| Consultar | DB |
| Conversar | conversacional |

---

## Prioridades:

- 🔴 HIGH → nuevas oportunidades
- 🟡 MEDIUM → enriquecimiento
- ⚪ LOW → limpieza

---

# 8. 🧠 PROMPT BASE DEL AGENTE

---

## SYSTEM PROMPT (CRÍTICO)

Eres AUKA-DIRECTOR, el agente principal de un sistema autónomo de prospección comercial.

Tu función es coordinar otros agentes especializados.

Reglas obligatorias:
1. NO ejecutes tareas directamente
2. SIEMPRE delega a otros agentes
3. SIEMPRE responde en formato JSON válido
4. NO inventes datos
5. PRIORIZA acciones que generen prospectos

Agentes disponibles:
- explorador
- scraper
- estructurador
- analista
- memoria

Flujo correcto:
usuario → análisis → decisión → delegación

Tu objetivo es maximizar la cantidad y calidad de prospectos.

---

# 9. 🔄 LOOP DE EJECUCIÓN

```
def director_loop(input):
    decision = LLM(input)

    for action in decision["actions"]:
        dispatch(action)

    results = collect_results()

    if needs_more(results):
        return director_loop(results)

    return results
```

---

# 10. 🚨 CONTROL DE ERRORES

---

## Casos:

### ❌ No hay resultados

→ cambiar estrategia

### ❌ Datos incompletos

→ enrichment

### ❌ duplicados

→ ignorar

---

## Ejemplo:

```
{
  "decision": "refinar busqueda",
  "actions": [
    {
      "agent": "explorador",
      "task": "search_instagram",
      "params": {
        "hashtag": "eventoscaracas"
      }
    }
  ]
}
```

---

# 11. 🧠 MEMORIA Y CONTEXTO

El Director usa:

- historial de búsquedas
- empresas ya vistas
- eventos detectados

---

## Ejemplo:

```
{
  "avoid": ["empresa_x", "evento_y"]
}
```

---

# 12. ⚡ OPTIMIZACIÓN

---

## Estrategias:

- evitar repetir queries
- priorizar fuentes activas
- cortar loops inútiles

---

# 13. 🔐 REGLAS CRÍTICAS (ANTI-ALUCINACIÓN)

1. Solo usar datos reales
2. No asumir información
3. No completar campos vacíos inventando
4. Validar antes de guardar

---

# 14. 🧪 CASOS DE USO REALES

---

## Caso 1:

INPUT:

> “dame nuevos prospectos en valencia”

OUTPUT:

```
{
  "actions": [
    {
      "agent": "explorador",
      "task": "search_maps",
      "params": {
        "query": "eventos valencia carabobo"
      }
    }
  ]
}
```

---

## Caso 2:

INPUT:

> “analiza esta empresa”

OUTPUT:

```
{
  "actions": [
    {
      "agent": "analista",
      "task": "evaluate_company",
      "params": {}
    }
  ]
}
```

---

# 15. 🚀 LISTO PARA PRODUCCIÓN

Este agente ya tiene:

✅ lógica clara
✅ formato estructurado
✅ control de flujo
✅ sistema de decisiones
✅ anti-alucinación

---

# 💬 SIGUIENTE PASO

Ahora lo más lógico es diseñar el siguiente agente:

👉 **Explorador (el que realmente encuentra oportunidades)**

Dime:

👉 “vamos con el explorador”

y lo hacemos igual de profundo (nivel producción).